3.11.2006

Operating Systems Midterm Exam
(2 hours)

Question 1: *(40 points)* Answer the following questions briefly.

a. What are the responsibilities of an operating system? Explain.
b. Define *processes* in the context of operating systems.
c. What are the advantages and disadvantages of using threads over multiple processes? Explain.
d. Define *busy-waiting*. When does busy-waiting occur? Explain.
e. What is a *critical section*? Explain through an example.
f. Define *system calls*. Why are system calls needed? Explain.
g. Define *time-sharing*. Why is it used? Explain.
h. What is *mutual exclusion*? When is it used? Explain.

Question 2: *(30 points)* Some special machine instructions exist which carry out two actions atomically, such as reading and writing or reading and testing of a single memory location in one instruction fetch cycle. Since these instructions are performed in a single instruction cycle, they are not subject to interference from other instructions. One such instruction is the *exchange* instruction, which is defined as follows:

```
void exchange(int register, int memory)
{
  int temp;
  temp=memory;
  memory=register;
  register=temp;
}
```

This instruction exchanges the contents of a register with that of a memory location. During the execution of this instruction, access to the memory location is blocked to all others. Below, a mutual exclusion protocol based on using this instruction is given. *Bolt* is a shared variable which is initialized to 0 and each process has a local variable called *key* which is initialized to 1. Discuss this hardware-based solution for mutual exclusion.

```
define N 10  //number of processes is 10
shared bolt;

P(i) {
  int keyi;
  while (true){
    keyi=1;
    while (keyi!=0)
      exchange(keyi,bolt);
/* Critical Section */
    exchange(keyi,bolt);
/* Non-critical Sections */
  }
}

main()
{
  bolt=0;
  parbegin(P(1), P(2), … , P(N));
}
```

Question 3: *(30 points)* Write the messages which will be output on the screen for each program given below. Explain your answer in detail.

Program P1

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int tmp=0;

void my_function(int count) {
   tmp=tmp+1;
   printf("%d: Value= %d\n", count, tmp);
}

int main(void)
{
  int f=1, i;

  for (i=0; i<3; i++) {
    if (f>0)
      f=fork();
    if (f==-1) {
      printf("fork error....\n");
      exit(-1);
    }
    if (f==0)
      break;
  }
  if (f == 0) {
     my_function(i);
  }
  else {
    printf("Main: Created %d procs.\n", i);
  }
  return 0;
}
```

Program P2

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int tmp=0;

void *my_function(void *threadid) {
  tmp=tmp+1;
  printf("%d: Value= %d\n", threadid, tmp);
  pthread_exit(NULL);
}

int main(void)
{
  pthread_t threads[3];
  int rc, i;

  for(i=0; i<3;i++) {
    rc = pthread_create(&threads[i], NULL,
                  my_function, (void *)i);
    if (rc) {
      printf("thread creation error ...\n);
      exit(-1);
    }
  }
  printf("main(): Created %d threads.\n", i);
  pthread_exit(NULL);
  return 0;
}
```